

TALLER ANALÍTICO: LA VOTACIÓN ESPACIAL (25 MIN)

Un sistema debe clasificar si un cliente comprará un producto basándose en dos características (X = Edad, Y = Salario en miles). Tenemos 3 clientes históricos (Dataset):

- Cliente 1: A(20, 30) → Clase: NO COMPRA
- Cliente 2: B(40, 50) → Clase: COMPRA
- Cliente 3: C(35, 45) → Clase: COMPRA

Llega un nuevo cliente desconocido: **Punto Nuevo (30, 40)**.

1. Calcule matemáticamente la Distancia Euclidiana desde el Punto Nuevo hasta A, B y C.
2. Si definimos **K = 1**, ¿cuál será la clasificación del nuevo cliente?
3. Si definimos **K = 3**, ¿cuál será la clasificación? ¿Hubo un cambio en la decisión?

1. La fórmula de distancia euclidiana es:

$$d(A, B) = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

Distancia de P a A

$$d(P, A) = \sqrt{[(30 - 20)^2 + (40 - 30)^2]}$$

$$d(P, A) = \sqrt{[(10)^2 + (10)^2]}$$

$$d(P, A) = \sqrt{[100 + 100]}$$

$$d(P, A) = \sqrt{200}$$

$$d(P, A) = 14.14$$

Distancia de P a B

$$d(P, B) = \sqrt{[(30 - 40)^2 + (40 - 50)^2]}$$

$$d(P, B) = \sqrt{[(-10)^2 + (-10)^2]}$$

$$d(P, B) = \sqrt{[100 + 100]}$$

$$d(P, B) = \sqrt{200}$$

$$d(P, B) = 14.14$$

Distancia de P a C

$$d(P, C) = \sqrt{[(30 - 35)^2 + (40 - 45)^2]}$$

$$d(P, C) = \sqrt{[(-5)^2 + (-5)^2]}$$

$$d(P, C) = \sqrt{[25 + 25]}$$

$$d(P, C) = \sqrt{50}$$

$$d(P, C) = 7.07$$

Resumen de distancias

$d(P, A) \approx 14.14$ (NO COMPRA)

$d(P, B) \approx 14.14$ (COMPRA)

$d(P, C) \approx 7.07$ (COMPRA) ← el más cercano

2. Con $K = 1$, solo miramos al vecino más cercano. Ese vecino es C, con una distancia de 7.07, como C pertenece a la clase compra, el nuevo cliente se clasifica como: compra
3. Con $K = 3$, tomamos en cuenta a los 3 vecinos
A → NO COMPRA (1 voto)
B → COMPRA (1 voto)
C → COMPRA (1 voto)
Gana la mayoría: compra
No, no hubo cambio. Tanto con $K=1$ como con $K=3$, el resultado es el mismo: COMPRA

TALLER DE LABORATORIO: CLASIFICADOR UNIVERSAL (35 MIN)

Misión Práctica:

1. Transcriban el código superior a su entorno Python.
2. Amplíen el dataset X_entrenamiento con al menos 10 puntos (filas) y agreguen una tercera columna (dimensión). Por ejemplo, (Edad, Salario, Número de Hijos).
3. Actualicen Y_entrenamiento para que coincida.
4. Experimenten con el valor de K. Entrenen el modelo con `n_neighbors=1`, pregunten por un punto nuevo, y luego cambien a `n_neighbors=5`.
5. **Pregunta de Análisis:** En Machine Learning existe "La Maldición de la Dimensionalidad". Si en lugar de 3 columnas tuvieran 1,000 columnas (como los píxeles de una imagen), ¿qué pasaría matemáticamente con la Distancia Euclidiana entre los puntos? Discutan en grupo y anoten su respuesta.

```

...
Bienvenido x clasificador_universal.py U x
Sesion 9 > clasificador_universal.py > ...
1  import numpy as np
2  from sklearn.neighbors import KNeighborsClassifier
3
4  X_entrenamiento = np.array([
5      [20, 30, 0],
6      [40, 50, 2],
7      [35, 45, 1],
8      [22, 28, 0],
9      [45, 60, 3],
10     [30, 40, 1],
11     [50, 70, 2],
12     [25, 32, 0],
13     [38, 48, 2],
14     [28, 35, 0],
15 ])
16
17 Y_entrenamiento = np.array([0, 1, 1, 0, 1, 1, 1, 0, 1, 0])
18
19 nuevo_cliente = np.array([[30, 40, 1]])
20
21 modelo_knn_k1 = KNeighborsClassifier(n_neighbors=1)
22 modelo_knn_k1.fit(X_entrenamiento, Y_entrenamiento)
23 prediccion_k1 = modelo_knn_k1.predict(nuevo_cliente)
24 print("Predicción con K=1:", prediccion_k1[0])
25
26 modelo_knn_k5 = KNeighborsClassifier(n_neighbors=5)
27 modelo_knn_k5.fit(X_entrenamiento, Y_entrenamiento)
28 prediccion_k5 = modelo_knn_k5.predict(nuevo_cliente)
29 print("Predicción con K=5:", prediccion_k5[0])

```



```
17 Y_entrenamiento = np.array([0. 1. 1. 0. 1. 1. 1. 0. 1. 0])

Problemas Salida Consola De Depuración Terminal Puertos

● @gomezzuley19-debug → /workspaces/IA-2-Talleres (main) $ source /workspaces/IA-2-Talleres/.venv/bin/activate
● (.venv) @gomezzuley19-debug → /workspaces/IA-2-Talleres (main) $ /workspaces/IA-2-Talleres/.venv/bin/python "/workspaces/I
Predicción con K=1: 1
Predicción con K=5: 1
○ (.venv) @gomezzuley19-debug → /workspaces/IA-2-Talleres (main) $
```

Pregunta de Análisis: La Maldición de la Dimensionalidad

Con 1,000 columnas, la distancia euclidiana se calcula igual, solo que sumando 1,000 términos en vez de 2 o 3

El problema es que, con tantas dimensiones, todos los puntos empiezan a quedar casi a la misma distancia entre sí. Ya no hay puntos claramente "cerca" y otros claramente "lejos", porque cada dimensión extra suma su propia diferencia, y al sumar tantas, todo se empareja.

Esto hace que KNN pierda su capacidad de distinguir quién es realmente parecido al nuevo punto, porque el concepto de "vecino más cercano" deja de tener sentido cuando todos están igual de lejos.

Conclusión: con muchas dimensiones (como los píxeles de una imagen), la distancia euclidiana deja de ser útil para medir similitud. Esto se llama Maldición de la Dimensionalidad, y por eso normalmente se reduce la cantidad de variables antes de usar KNN o se usan otros algoritmos mejores para esos casos